

EMBEDDED RETRO

Back to Fundamentals

แผนการเรียนรู้พื้นฐาน

ย้อนกลับสู่รากฐานของ Embedded Systems, Firmware, Electronics และ Computer Engineering



ระบบฝังตัว



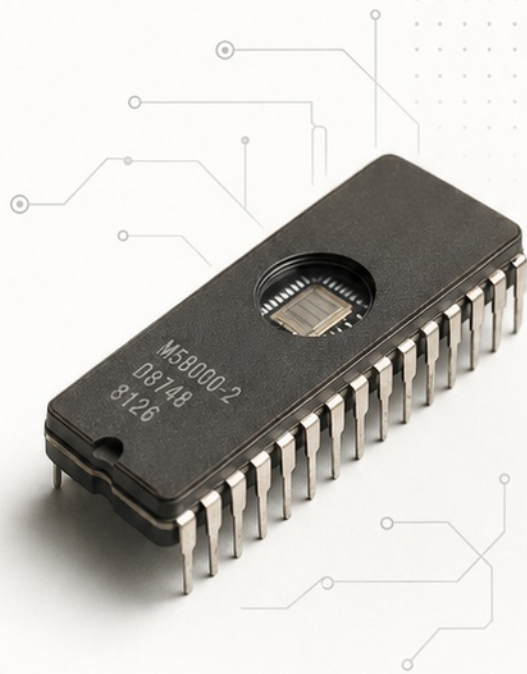
เฟิร์มแวร์



อิเล็กทรอนิกส์



วิศวกรรมคอมพิวเตอร์



หัวข้อทั้งหมด

1 / 10

หน้าแรก



0 1

1

บิต ไบต์ และระบบเลข

เข้าใจตัวเลขฐานต่าง ๆ และการแทนข้อมูล



2

ลอจิกเกต และพีชคณิตบูลีน

พื้นฐานของตรรกะดิจิทัลและการคำนวณ



3

วงจรคอมบินেশัน และซีเควนเชียล

การออกแบบวงจรตรรกะและลำดับสถานะ



4

CPU และสถาปัตยกรรมพื้นฐาน

โครงสร้าง CPU และการทำงานภายใน



5

หน่วยความจำ และระบบบัส

ชนิดของหน่วยความจำและการสื่อสารบนบัส



6

CPU คลาสสิก: 8085, Z80, 8086

ศึกษาสถาปัตยกรรมและคุณสมบัติของซีพียูยุคต้น



7

ไมโครคอนโทรลเลอร์คลาสสิก: 8051

สถาปัตยกรรม 8051 และการใช้งานพื้นฐาน



8

ภาษาเครื่อง และแอสเซมบลี

รหัสเครื่องและการเขียนโปรแกรมระดับต่ำ



9

Embedded C และเฟิร์มแวร์

เขียนโปรแกรม C บนระบบฝังตัว และการพัฒนาเฟิร์มแวร์



10

มาตรฐานโค้ด และการพัฒนาแบบมืออาชีพ

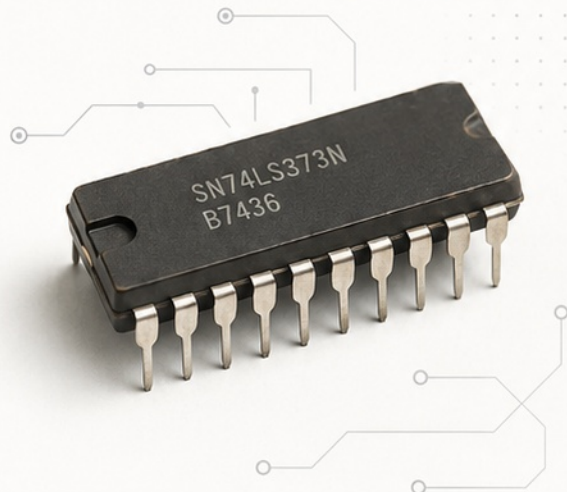
แนวทางโค้ดที่ดี เครื่องมือ และกระบวนการพัฒนา



บิต ไบต์ และระบบเลข

พื้นฐานของข้อมูลในระบบดิจิทัล

คอมพิวเตอร์ยุคแรกของเราใช้ข้อมูลในรูปแบบตัวเลขฐานสอง ทุกสิ่งถูกเก็บและประมวลผลด้วยบิตและไบต์



1 บิต (Bit)

หน่วยข้อมูลที่เล็กที่สุด มีได้ 2 ค่าเท่านั้น

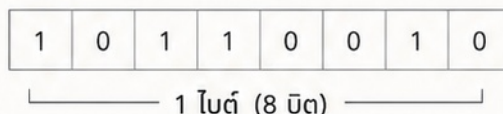
0 = LOW

1 = HIGH



2 ไบต์ (Byte)

8 บิต = 1 ไบต์



3 เลขฐานสอง (Binary)

แต่ละตำแหน่งมีค่าน้ำหนักเป็น 2 ยกกำลัง

128	64	32	16	8	4	2	1
2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0

4 ตัวอย่างการแปลง

ตัวอย่าง

0	1	0	1	0	1	0	1
---	---	---	---	---	---	---	---

$$0 \times 128 + 1 \times 64 + 0 \times 32 + 1 \times 16 + 0 \times 8 + 1 \times 4 + 0 \times 2 + 1 \times 1$$

$$= 85 \text{ (ฐานสิบ)} = 55H \text{ (ฐานสิบหก)}$$

5 เลขฐานสิบหก (Hexadecimal)

ใช้ตัวเลข 16 ตัว: 0-9, A-F

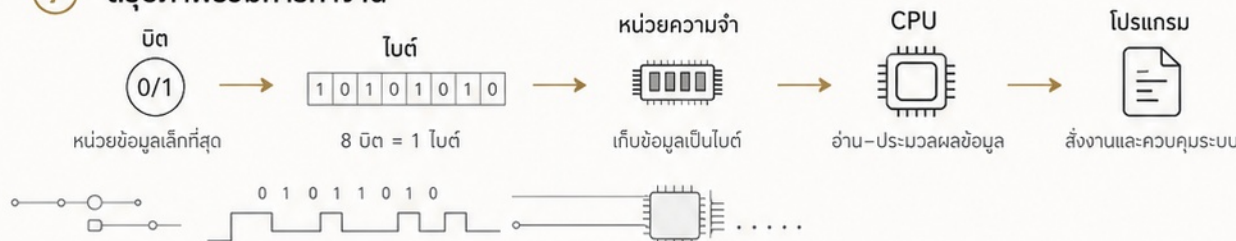
0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

1 หลัก Hex = 4 บิต | 2 หลัก Hex = 1 ไบต์

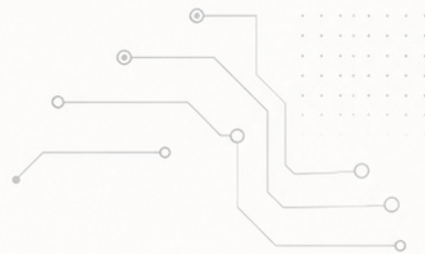
6 ตัวอย่างแบบ Retro

8085	8085: A = 55H	รีจิสเตอร์ A เก็บค่าไบต์ 55H (0101 0101)
8051	8051: P1 = 01H	พอร์ต P1 มีค่า 01H (0000 0001)

7 สรุปภาพรวมการทำงาน

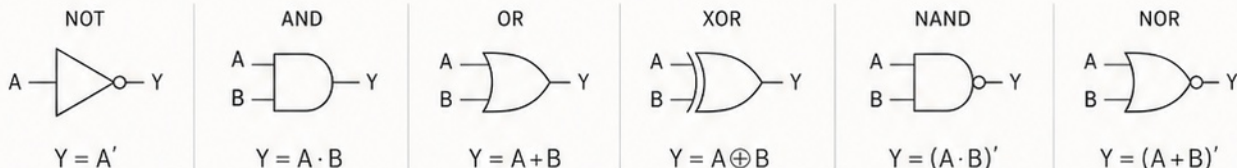


ลอจิกเกต และพีชคณิตบูลีน



ภาษาของวงจรดิจิทัล

1 ลอจิกเกตพื้นฐาน



2 ตารางความจริง (Truth Table)

A	B	A'	A · B	A + B	A ⊕ B	(A · B)'	(A + B)'
0	0	1	0	0	0	1	1
0	1	1	0	1	1	1	0
1	0	0	0	1	1	1	0
1	1	0	1	1	0	0	0

3 สัญลักษณ์พีชคณิตบูลีน

- $A \cdot B$ AND (คูณตรรกะ)
เป็นจริงเมื่อ A และ B เป็นจริง
- $A + B$ OR (บวกตรรกะ)
เป็นจริงเมื่อ A หรือ B อย่างน้อยหนึ่งตัวเป็นจริง
- A' NOT (การกลับค่า)
สลับค่า 0 เป็น 1 และ 1 เป็น 0

4 กฎของเดอมอร์แกน (De Morgan's Law)

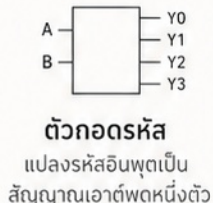
$$(A + B)' = A' B'$$

นิเสธผลรวม = ผลคูณของนิเสธ

$$(A \cdot B)' = A' + B'$$

นิเสธผลคูณ = ผลรวมของนิเสธ

5 การประยุกต์ใช้งาน



6 ข้อควรรู้

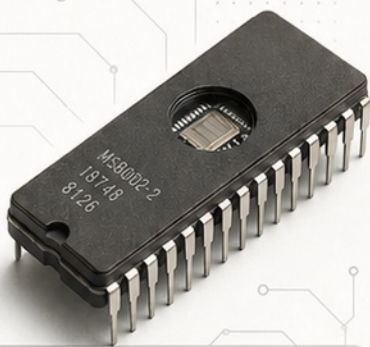


ลอจิกเกต คือหน่วยพื้นฐานของ CPU และอิเล็กทรอนิกส์ดิจิทัล



วงจรคอมบิเนชัน และซีเควนเชียล

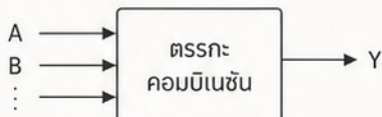
ตรรกะที่ไม่มีหน่วยจำ และตรรกะที่มีสถานะ



1. เปรียบเทียบแนวคิด

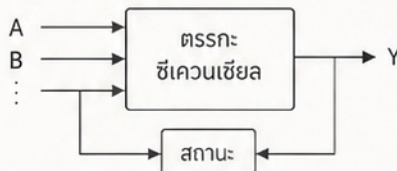
คอมบิเนชัน

เอาต์พุตขึ้นอยู่กับอินพุตปัจจุบันเท่านั้น
ไม่มีหน่วยจำ หรือสถานะ

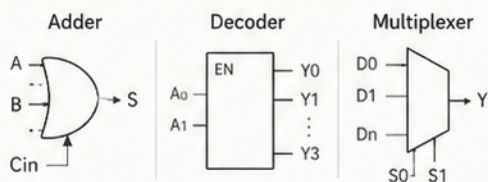


ซีเควนเชียล

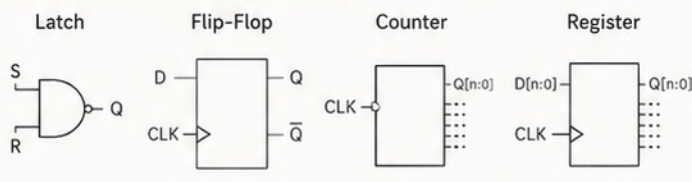
เอาต์พุตขึ้นอยู่กับอินพุตปัจจุบัน
และสถานะก่อนหน้า



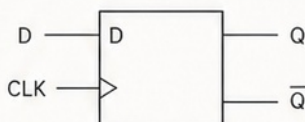
2. ตัวอย่างวงจรคอมบิเนชัน



3. ตัวอย่างวงจรซีเควนเชียล



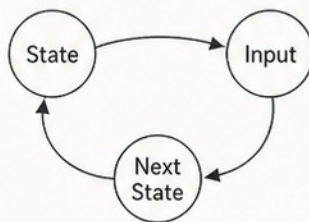
4. D Flip-Flop (ตัวอย่างพื้นฐานซีเควนเชียล)



เมื่อสัญญาณนาฬิกา
(CLK) ขึ้นขอบบวก (↑)
เอาต์พุต Q จะรับค่า
จากอินพุต D

D	CLK ↑	Q(t+1)
0	↑	0
1	↑	1

5. State Machine อย่างง่าย



สถานะปัจจุบัน (State)
รับอินพุต (Input)
และให้สถานะถัดไป
(Next State)

6. การใช้งานของวงจรซีเควนเชียล



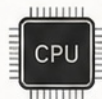
ตัวนับเวลา

นับเวลา, จับเวลา, แสดงผล



UART

การสื่อสารอนุกรม
รับ-ส่งข้อมูล



รีจิสเตอร์ของ CPU

เก็บข้อมูลภายใน
และสถานะต่าง ๆ



วงจรควบคุม

ควบคุมลำดับการทำงาน
ของระบบ



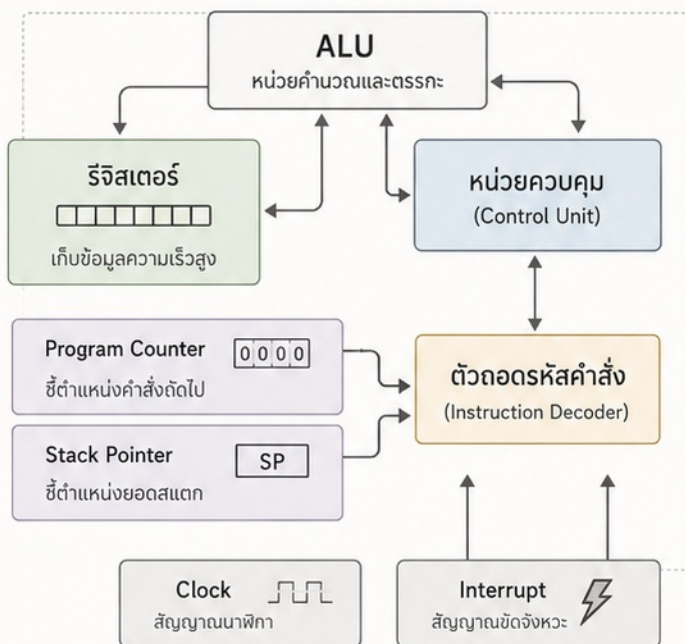
CPU และสถาปัตยกรรมพื้นฐาน

มองภายในหน่วยประมวลผล

CPU (Central Processing Unit) คือ “สมอง” ของระบบ ทำหน้าที่ดึงคำสั่ง ประมวลผล และควบคุมการทำงานของทุกส่วนในระบบคอมพิวเตอร์



สถาปัตยกรรม CPU แบบคลาสสิก



วงจรการทำงานพื้นฐาน



หน่วยข้อมูล	เก็บข้อมูลและผลลัพธ์ชั่วคราว
หน่วยควบคุม	ควบคุมลำดับการทำงาน
ถอดรหัส	แปลคำสั่งให้หน่วยควบคุมเข้าใจ
ตัวชี้ตำแหน่ง	บอกตำแหน่งในหน่วยความจำ
สัญญาณควบคุม	นาฬิกาและสัญญาณขัดจังหวะ

รีจิสเตอร์คือหน่วยเก็บข้อมูลความเร็วสูง

รีจิสเตอร์อยู่ภายใน CPU ทำงานเร็วมาก ใช้เก็บข้อมูล, คำสั่ง, ที่อยู่, ผลลัพธ์ชั่วคราว และธงสถานะ (Flags)



บัสเชื่อมต่อกับหน่วยความจำและ I/O

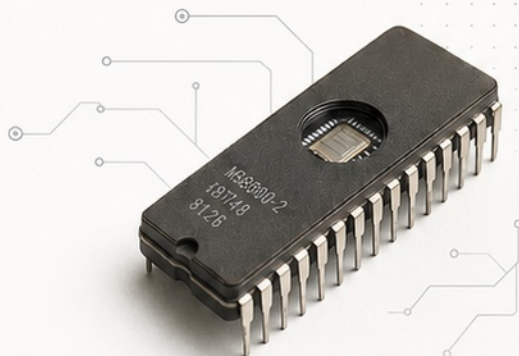
CPU ใช้บัสเป็นเส้นทางในการรับ-ส่งข้อมูลกับหน่วยความจำและอุปกรณ์ I/O ภายนอก



หน่วยความจำ และระบบบัส

6 / 10 หน้า 6

ที่เก็บโปรแกรมและเส้นทางรับส่งข้อมูล



1) ประเภทของหน่วยความจำ

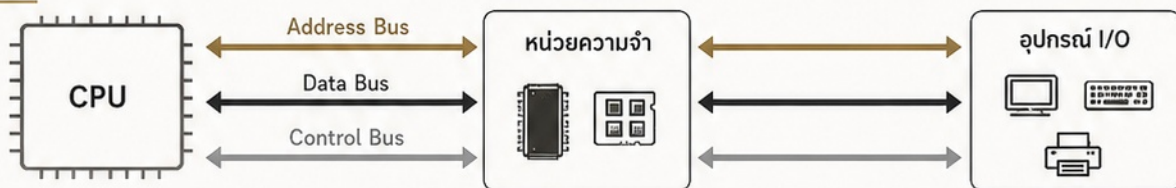
	ROM หน่วยความจำแบบอ่านอย่างเดียว เก็บโปรแกรมถาวร
	RAM หน่วยความจำแบบอ่าน/เขียน ใช้เก็บข้อมูลชั่วคราว
	EPROM ลบด้วยแสงอัลตราไวโอเลต แล้วเขียนใหม่ได้
	EEPROM ลบและเขียนใหม่ได้ด้วยไฟฟ้า ไม่ต้องถอดชิป

2) ตัวอย่างแผนที่หน่วยความจำ

ช่วงแอดเดรส (Hex)	อุปกรณ์/หน่วยความจำ
0000H - 1FFFFH	ROM (โปรแกรมระบบ)
2000H - 27FFFH	RAM (พื้นที่ข้อมูล)
3000H - 30FFFH	I/O (พอร์ตอุปกรณ์)

0000H FFFFH

3) โครงสร้างระบบบัส



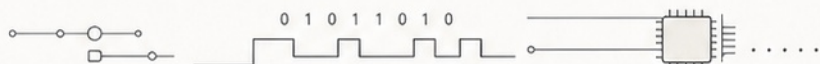
4) สัญญาณควบคุม (ตัวอย่าง)

	RD (Read)	สั่งให้อ่านข้อมูลจากหน่วยความจำหรืออุปกรณ์
	WR (Write)	สั่งให้เขียนข้อมูลไปยังหน่วยความจำหรืออุปกรณ์
	CS (Chip Select)	เลือกหน่วยความจำหรืออุปกรณ์ที่ต้องการใช้งาน
	RESET	รีเซ็ตระบบ กลับสู่สถานะเริ่มต้น

5) โน้ตสำคัญ

 **Address Bus**
ใช้ระบุตำแหน่ง

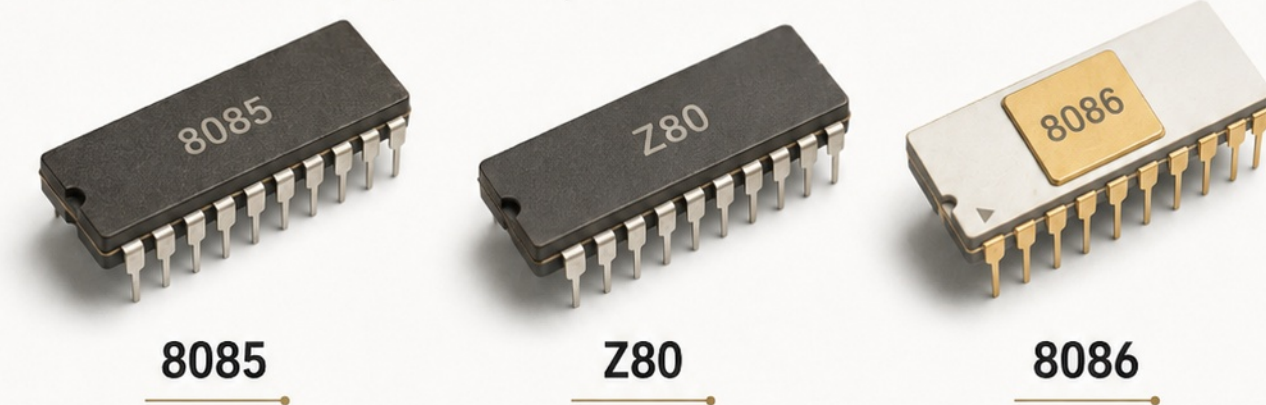
 **Data Bus**
ใช้ส่งข้อมูล



CPU คลาสสิก: 8085, Z80, 8086

เรียนรู้พื้นฐานจากซีพียูยุคบุกเบิก

ซีพียูรุ่นสำคัญที่วางรากฐานให้กับคอมพิวเตอร์
ไมโครคอนโทรลเลอร์ และสถาปัตยกรรมร่วมสมัย
มาทำความเข้าใจความแตกต่าง และจุดเด่นของแต่ละรุ่น



	สถาปัตยกรรม	8 บิต	8 บิต	16 บิต
	แอดเดรสบัส	16 บิต	16 บิต	20 บิต
	ช่วงหน่วยความจำ	64KB	64KB	1MB
	เหมาะกับการเรียนรู้	เรียนรีจิสเตอร์ และอินเทอร์พรัพท์	ชุดคำสั่งมากขึ้น ใช้แพร่หลายใน CP/M เหมาะกับแอสเซมบลี	จุดเริ่มต้นของ สถาปัตยกรรม x86

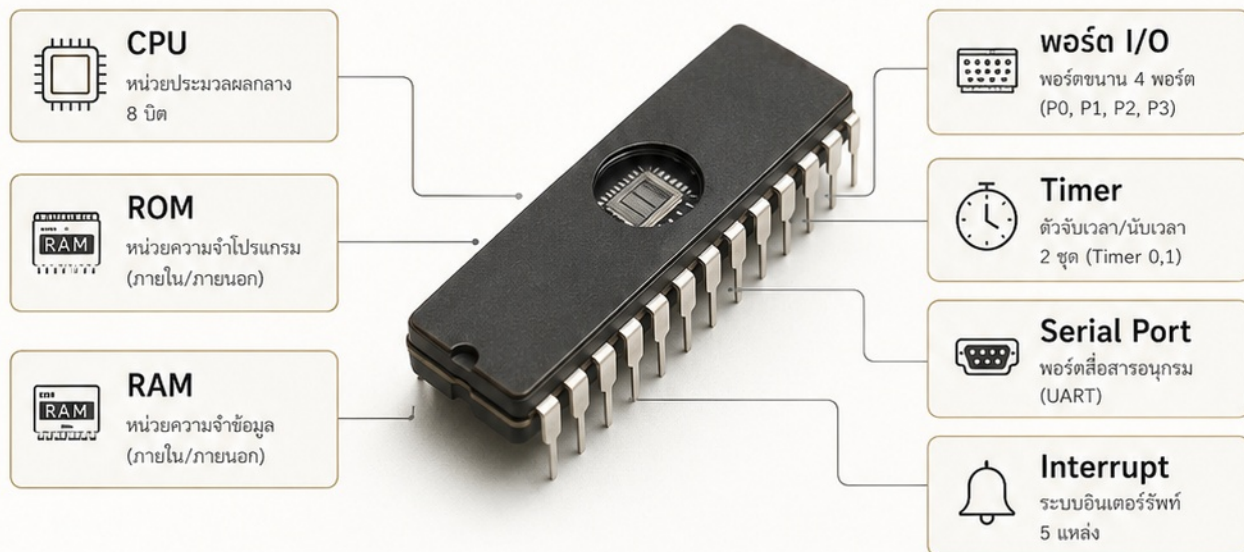
ไทม์ไลน์การเปิดตัว



ไมโครคอนโทรลเลอร์

คลาสสิก: 8051

คอมพิวเตอร์หนึ่งชิปในตำนาน



1 สิ่งที่ต้องรู้

พอร์ต 0-3
พอร์ตขนาน 8 บิต 4 พอร์ต
ใช้รับ-ส่งข้อมูลกับอุปกรณ์ภายนอก

Timer 0/1
จับเวลา นับเวลา สร้างหน่วยเวลา
หรือสร้างสัญญาณเป็นคาบ

UART
สื่อสารอนุกรมแบบ Full-Duplex
ใช้ RXD (P3.0) และ TXD (P3.1)

SFR (Special Function Register)
กลุ่มรีจิสเตอร์ควบคุมการทำงานของฮาร์ดแวร์ภายใน

2 ตัวอย่างโค้ดแอสเซมบลี

```
MOV A, #55H ; โหลดค่า 55H ลงใน A
MOV P1, A ; ส่งค่าใน A ออกพอร์ต P1
SJMP $ ; วนลูปที่ตำแหน่งเดิม
```

```
MOV A, #55H : โหลดค่าคงที่ 55H เข้าสะสม A
MOV P1, A : ส่งค่าจาก A ออกพอร์ต P1
SJMP $ : กระโดดไปตำแหน่งเดิม (ลูปไม่สิ้นสุด)
```

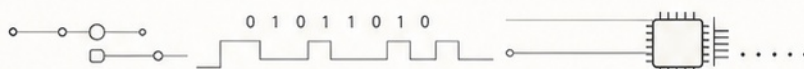
หมายเหตุ
เป็นตัวอย่างพื้นฐานที่ใช้ในการทดสอบพอร์ตเอาต์พุต

3 การใช้งาน

การศึกษา
สอนพื้นฐานไมโครคอนโทรลเลอร์
และระบบฝังตัว

ระบบควบคุม
ควบคุมอุปกรณ์ตามเวลา
รับสัญญาณและสั่งงาน

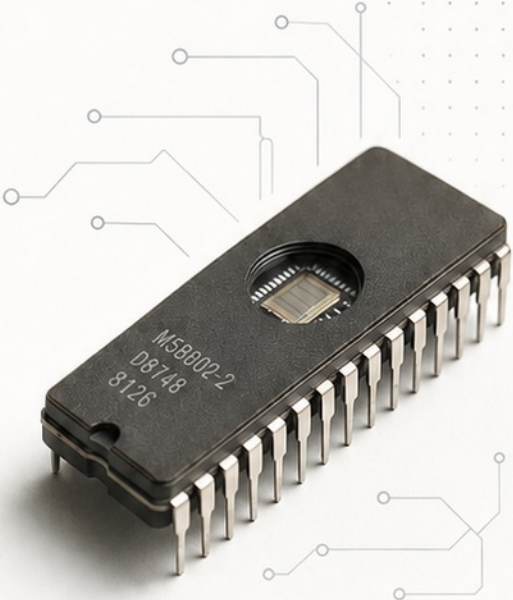
งานอุตสาหกรรม
ใช้งานในเครื่องจักรและอุปกรณ์
ที่ต้องการความทนทาน



ภาษาเครื่อง และแอสเซมบลี

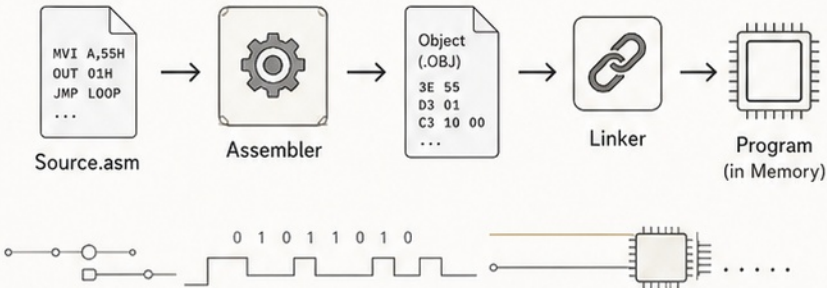
จากรหัสเครื่องสู่คำสั่งที่มนุษย์อ่านได้

คอมพิวเตอร์เข้าใจเฉพาะรหัสเครื่อง (Machine Code) ซึ่งเป็นเลขฐานสองเท่านั้น เราจึงใช้ภาษาแอสเซมบลี (Assembly Language) เป็นภาษาย่อที่อ่านและเขียนง่ายขึ้น



1		Machine Code รหัสเครื่อง	คอมพิวเตอร์เข้าใจเฉพาะตัวเลขฐานสอง หรือเลขฐานสิบหก (Hex)	<table><tr><td>01010101</td><td>11000001</td><td>00000001</td></tr><tr><td>55H</td><td>C1H</td><td>01H</td></tr></table>	01010101	11000001	00000001	55H	C1H	01H		
01010101	11000001	00000001										
55H	C1H	01H										
2		Opcode = รหัสคำสั่ง	ส่วนของคำสั่งที่บอกว่า ต้องทำอะไร	Opcode (Operation Code) กำหนดการทำงาน เช่น MOV, OUT, JMP								
3		Assembly = ภาษาย่อที่อ่านง่ายขึ้น	ใช้คำนำ (Mnemonic) แทนรหัส และตามด้วยตัวดำเนินการ (Operand)	<table><tr><th>ตัวอย่าง 8085</th><th>ตัวอย่าง 8051</th></tr><tr><td>MVI A, 55H</td><td>MOV A, #55H</td></tr><tr><td>OUT 01H</td><td>MOV P1, A</td></tr><tr><td>JMP LOOP</td><td>SJMP LOOP</td></tr></table>	ตัวอย่าง 8085	ตัวอย่าง 8051	MVI A, 55H	MOV A, #55H	OUT 01H	MOV P1, A	JMP LOOP	SJMP LOOP
ตัวอย่าง 8085	ตัวอย่าง 8051											
MVI A, 55H	MOV A, #55H											
OUT 01H	MOV P1, A											
JMP LOOP	SJMP LOOP											
4		Assembler = แปลง Assembly เป็น Object Code	แปลคำสั่งแอสเซมบลีเป็นรหัสเครื่อง และสร้างไฟล์อ็อบเจกต์ (.OBJ)	Object Code (รหัสเครื่อง) <table><tr><td>3E 55</td></tr><tr><td>D3 01</td></tr><tr><td>C3 10 00</td></tr></table>	3E 55	D3 01	C3 10 00					
3E 55												
D3 01												
C3 10 00												
5		Linker / Loader = รวมและนำโปรแกรม เข้าสู่หน่วยความจำ	รวมอ็อบเจกต์หลายไฟล์ จัดการที่อยู่ และโหลดโปรแกรมเข้าสู่ RAM/ROM เพื่อให้ CPU เริ่มทำงาน	Program in Memory <table><tr><td>2000:</td><td>3E 55</td></tr><tr><td>2002:</td><td>D3 01</td></tr><tr><td>2004:</td><td>C3 10 00</td></tr><tr><td>...</td><td></td></tr></table>	2000:	3E 55	2002:	D3 01	2004:	C3 10 00	...	
2000:	3E 55											
2002:	D3 01											
2004:	C3 10 00											
...												

● Pipeline การทำงาน



 **หมายเหตุ**

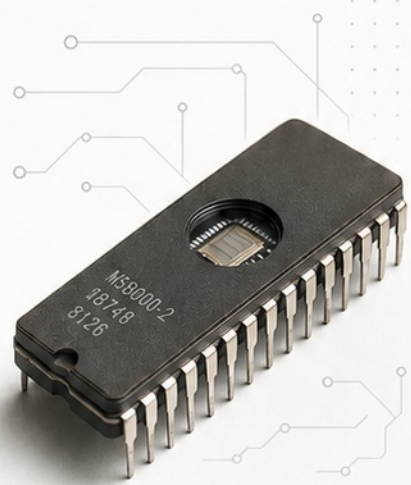
Mnemonic (คำนำ)
คำย่อที่จดจำได้ง่าย เช่น MOV, OUT, JMP

Operand (ตัวดำเนินการ)
ข้อมูล, รีจิสเตอร์ หรือแอดเดรส ที่ใช้ร่วมกับคำสั่ง

EMBEDDED C

และมาตรฐานโค้ด

จากเฟิร์มแวร์พื้นฐานสู่การพัฒนาแบบมืออาชีพ



1 Embedded C

10 / 10 หน้าสุดท้าย

🔲	🔄	1	โครงสร้างหลัก (Main Loop)	ลูปหลักไม่สิ้นสุด ควบคุมการทำงานทั้งหมดของระบบ
○	🧩	2	การแยกฟังก์ชัน (Function Decomposition)	แบ่งงานออกเป็นฟังก์ชันย่อย เพื่อความชัดเจนและนำกลับมาใช้ซ้ำได้
○	👤	3	เครื่องจักรสถานะ (State Machine)	จัดการพฤติกรรมระบบตามสถานะอย่างเป็นระบบ
○	🏠	4	บิตแมสก์ (Bit Mask)	จัดการบิตและแฟล็กอย่างมีประสิทธิภาพและประหยัดทรัพยากร
○	volatile	5	ตัวแปรแบบ Volatile	ใช้กับข้อมูลที่เปลี่ยนแปลงโดยฮาร์ดแวร์หรืออินเทอร์รัพต์
○	*	6	พอยน์เตอร์ (Pointer)	เข้าถึงหน่วยความจำและอาร์เรย์อย่างยืดหยุ่นและมีประสิทธิภาพ
○	⚡	7	อินเทอร์รัพต์เซอร์วิสรูทีน (ISR)	ตอบสนองเหตุการณ์ทันที รวดเร็ว และปลอดภัย
○	{ }	8	ตัวอย่างโค้ดหลัก	<pre>while(1) { scan(); update(); output(); }</pre>

2 มาตรฐานและคุณภาพโค้ด

○	🛡️	1	MISRA C	แนวทางมาตรฐานสำหรับโค้ด C ที่ปลอดภัยและเชื่อถือได้
○	🛡️	2	CERT C	แนวทางลดช่องโหว่และนิกด้านความปลอดภัยในภาษา C
○	🔍	3	Static Analysis	ตรวจจับข้อผิดพลาดและปัญหาคุณภาพโค้ดโดยอัตโนมัติ
○	👥	4	Code Review	ตรวจสอบโค้ดโดยทีม เพื่อคุณภาพและการเรียนรู้ร่วมกัน
○	✅	5	Unit Test	ทดสอบหน่วยฟังก์ชันย่อย ยืนยันผลลัพธ์ที่ถูกต้องและเสถียร
○	📖	6	Documentation	บันทึกการออกแบบ การใช้งาน และข้อจำกัดอย่างครบถ้วน
○	🔗	7	Version Control	ควบคุมเวอร์ชันและประวัติการเปลี่ยนแปลงโค้ดอย่างเป็นระบบ

★ โค้ดที่ดีต้องอ่านง่าย ปลอดภัย และดูแลรักษาได้

